Lab 08 – Worksheet

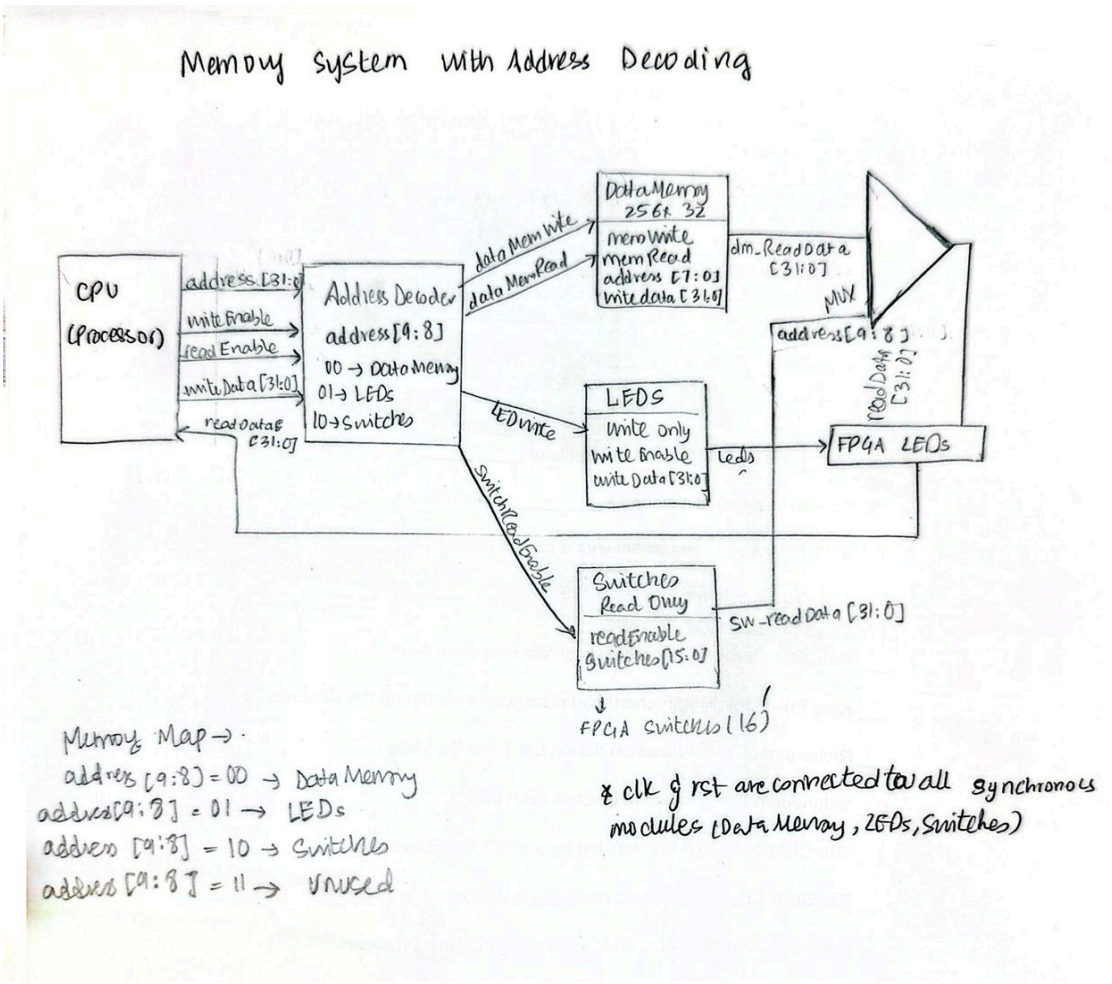
Name: Myrah Hussain Romaisa Shazad	ID: mh10074 rb10107	Section: L3

Note: Assumptions and logics should be explained separately in tasks after the task results.

Task 1

Block Diagram
a) Data Memory Module
b) Address Decoder
c) Top Module

Block diagram:



Memory Map →
address [9:8] = 00 → Data Memory
address [9:8] = 01 → LEDs
address [9:8] = 10 → Switches
address [9:8] = 11 → unused

* clk & rst are connected to all synchronous modules (Data Memory, LEDs, Switches)

Comments

Task 2

Modules for ***DataMemory, AddressDecoder, AddressDecoderTop, Switches and Leds.***

a) Data Memory Module

The `DataMemory` module implements a 512 × 32-bit data memory. Writes are synchronous — data is stored on the rising clock edge when `memWrite` is asserted. A synchronous reset clears all 512 locations to zero. Reads are asynchronous — `readData` continuously reflects the value stored at the given address without waiting for a clock edge. The 8-bit address input allows direct indexing of all 512 locations, corresponding to the `address[7:0]` field assigned to data memory in the system memory map (`address[9:8] = 00`).

```
`timescale 1ns / 1ps

// Data Memory Module
// Implements a 512-location x 32-bit synchronous write, asynchronous read
memory.
// module DataMemory(
   input clk,          // System clock
   input rst,          // Synchronous reset — clears all memory to 0
   input [7:0] address, // 8-bit address — selects one of 512 locations
   input memWrite,     // Write enable — high to store writeData into memory
   input [31:0] writeData,  // 32-bit data to write
   output [31:0] readData   // 32-bit data read out (asynchronous)
);

   // 512 x 32-bit memory array
   reg [31:0] memory [0:511];
   integer i;

   // Synchronous write and reset
   // On reset: all 512 locations cleared to 0
   // On memWrite: store writeData at the given address
   always @(posedge clk) begin
      if (rst) begin
         for (i = 0; i < 512; i = i + 1)
            memory[i] <= 32'b0;
      end else if (memWrite) begin
         memory[address] <= writeData;
      end
```

```
    end

    // Asynchronous read — always outputs current value at address,
    // no clock edge required
    assign readData = memory[address];

endmodule
```

b) Address Decoder

The `AddressDecoder` module looks at the top 2 bits of the address (`address[9:8]`) to decide which device the CPU is talking to. It passes the CPU's read or write signal through to only that one device, keeping all other devices off. This ensures that only one device responds to any given memory access.

```
`timescale 1ns / 1ps

// Address Decoder Module
// Decodes address[9:8] to enable exactly one device at a time.
// Prevents bus contention by ensuring only one enable signal is high per access.
module AddressDecoder(
    input [1:0] address,      // address[9:8] — selects the target device
    input readEnable,         // From CPU — read request
    input writeEnable,        // From CPU — write request
    output dataMemWrite,      // Enables data memory write  (address = 00)
    output dataMemRead,       // Enables data memory read   (address = 00)
    output LEDWrite,          // Enables LED write          (address = 01)
    output switchReadEnable   // Enables switch read        (address = 10)
);
    // Each output is only active when the correct device is selected
    // AND the corresponding read/write enable from the CPU is asserted
    assign dataMemWrite     = (address == 2'b00) ? writeEnable : 1'b0;
    assign dataMemRead      = (address == 2'b00) ? readEnable  : 1'b0;
    assign LEDWrite         = (address == 2'b01) ? writeEnable : 1'b0;
    assign switchReadEnable = (address == 2'b10) ? readEnable  : 1'b0;

endmodule
```

c) *Address Decoder Top*

`AddressDecoderTOP` is the top-level memory system module. It instantiates the address decoder, data memory, LED interface, and switch interface, and wires them all together. The address decoder looks at `address[9:8]` and enables the correct device. Write data from the CPU is passed to the selected device, and read data is

returned to the CPU through a multiplexer that selects between data memory and switches depending on the address.

```
`timescale 1ns / 1ps

// Address Decoder Top Module
// Connects the CPU to all three devices: Data Memory, LEDs, and Switches.
// Instantiates the address decoder and routes data to the correct device.
module AddressDecoderTOP(
    input clk,
    input rst,
    input [31:0] address,       // full 32-bit CPU address
    input readEnable,           // CPU wants to read
    input writeEnable,          // CPU wants to write
    input [31:0] writeData,     // data from CPU to write
    input [15:0] switches_in,   // physical switch inputs
    output [31:0] readData,     // data returned to CPU
    output [15:0] leds_out      // drives physical LEDs
);
    // Internal enable signals from decoder
    wire dataMemWrite, dataMemRead, LEDWrite, switchReadEnable;
    // Read data wires from each device
    wire [31:0] dm_readData, sw_readData;

    // Decode address[9:8] to generate device enable signals
    AddressDecoder ad (
        .address(address[9:8]),
        .readEnable(readEnable),
        .writeEnable(writeEnable),
        .dataMemWrite(dataMemWrite),
        .dataMemRead(dataMemRead),
        .LEDWrite(LEDWrite),
        .switchReadEnable(switchReadEnable)
    );

    // Data Memory — uses lower 8 bits as local address
    DataMemory dataMem (
        .clk(clk),
        .rst(rst),
        .address(address[7:0]),
        .memWrite(dataMemWrite),
        .writeData(writeData),
        .readData(dm_readData)
    );

    // LED interface — write only, readEnable tied low
```

```
    leds led_inst (
        .clk(clk),
        .rst(rst),
        .writeData(writeData),
        .writeEnable(LEDWrite),
        .readEnable(1'b0),          // LEDs are write-only in this system
        .memAddress(address[31:2]),     // pass upper bits as memAddress [29:0]
        // readData output unused
        .leds(leds_out)
    );

    // Switch interface — read only, write signals tied low
    switches sw_inst (
        .clk(clk),
        .rst(rst),
        .btns(16'b0),               // no physical buttons used here
        .writeData(32'b0),          // switches are read-only
        .writeEnable(1'b0),
        .readEnable(switchReadEnable),
        .memAddress(address[31:2]),
        .switches(switches_in),
        .readData(sw_readData)
    );

    // Return data from the selected device back to CPU
    // Only data memory and switches can be read
    assign readData = (address[9:8] == 2'b00) ? dm_readData :
                      (address[9:8] == 2'b10) ? sw_readData :
                      32'b0;
endmodule
```

d) LEDs and Switches

The `leds` module stores a 16-bit value written by the CPU and drives it directly to the physical LEDs. On a write, the lower 16 bits of `writeData` are latched on the rising clock edge. The `switches` module captures the 16-bit physical switch inputs and returns them to the CPU as a 32-bit zero-extended value when `readEnable` is asserted. Both modules are synchronous with an active-high reset, and include unused ports for compatibility with the Lab 5 interface.

```
`timescale 1ns/1ps
module leds(
    input clk,
    input rst,
    input [31:0] writeData,
```

```
    input writeEnable,
    input readEnable,
    input [29:0] memAddress,
    output reg [31:0] readData = 0,
    output reg [15:0] leds
);
always @(posedge clk or posedge rst)
begin
    if (rst)
        leds <= 16'b0;
    else if (writeEnable)
        leds <= writeData[15:0];
end
endmodule

`timescale 1ns/1ps
module switches(
    input clk, rst,
    input [15:0] btns,
    input [31:0] writeData,
    input writeEnable,
    input readEnable,
    input [29:0] memAddress,
    input [15:0] switches,
    output reg [31:0] readData
);
always @(posedge clk or posedge rst)
begin
    if (rst)
        readData <= 32'b0;
    else if (readEnable)
        readData <= {16'b0, switches};
end
endmodule
```

Testbench:

The testbench instantiates `AddressDecoderTOP` and applies a sequence of test cases to verify correct behaviour across all devices. It tests writing and reading data memory, writing to LEDs, and reading switches with different input values. Cross-talk tests confirm that writing to one device does not affect another — a DM write must not change LED output and vice versa. Reset tests verify that writes are blocked while reset is asserted and that all outputs return to zero. The `$monitor` statement logs every signal change to the console for easy verification.

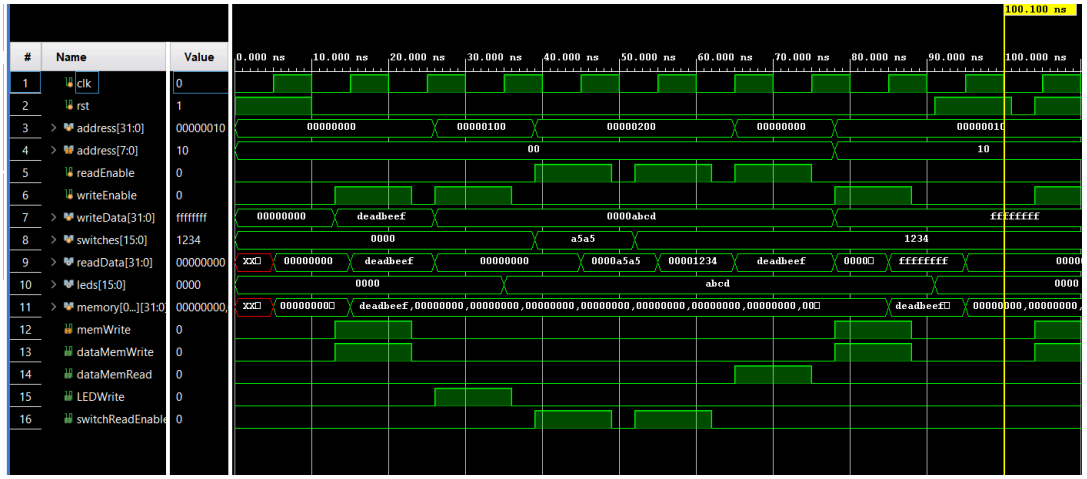
```
`timescale 1ns / 1ps

// Testbench for AddressDecoderTOP
// Verifies data memory, LED, and switch operations including reset behaviour
// and cross-talk checks (writes to one device should not affect others)
module MemorySystem_tb;
    reg clk;
    reg rst;
    reg [31:0] address;
    reg readEnable;
    reg writeEnable;
    reg [31:0] writeData;
    reg [15:0] switches;
    wire [31:0] readData;
    wire [15:0] leds;

    // 10ns clock period (100 MHz)
    always #5 clk = ~clk;

    // Device under test
    AddressDecoderTOP dut (
        .clk(clk),
        .rst(rst),
        .address(address),
        .readEnable(readEnable),
        .writeEnable(writeEnable),
```

```
        .writeData(writeData),
        .switches_in(switches),
        .readData(readData),
        .leds_out(leds)
    );

    initial begin
        // Initialise all signals and hold reset high
        clk = 0; rst = 1;
        address = 0; readEnable = 0; writeEnable = 0;
        writeData = 0; switches = 0;

        // Release reset
        #10 rst = 0;

        // Test 1: Write 0xDEADBEEF to data memory address 0x00
        #3  address = 32'h00000000; writeData = 32'hDEADBEEF; writeEnable = 1;
        #10 writeEnable = 0;

        // Test 4a: Write 0xABCD to LED interface
        #3  address = 32'h00000100; writeData = 32'h0000ABCD; writeEnable = 1;
        #10 writeEnable = 0;

        // Test 5a: Read switches with value 0xA5A5
        #3  address = 32'h00000200; switches = 16'hA5A5; readEnable = 1;
        #10 readEnable = 0;

        // Test 5b: Read switches with new value 0x1234 — confirms live input
captured
        #3  address = 32'h00000200; switches = 16'h1234; readEnable = 1;
        #10 readEnable = 0;

        // Test 6: Read data memory — should still be 0xDEADBEEF (LED write
must not affect DM)
        #3  address = 32'h00000000; readEnable = 1;
        #10 readEnable = 0;

        // Test 7: Write to a different DM address — LEDs must remain unchanged
        #3  address = 32'h00000010; writeData = 32'hFFFFFFFF; writeEnable = 1;
        #10 writeEnable = 0;

        // Test 8: Assert reset — clears memory and LEDs
        #3  rst = 1;
```

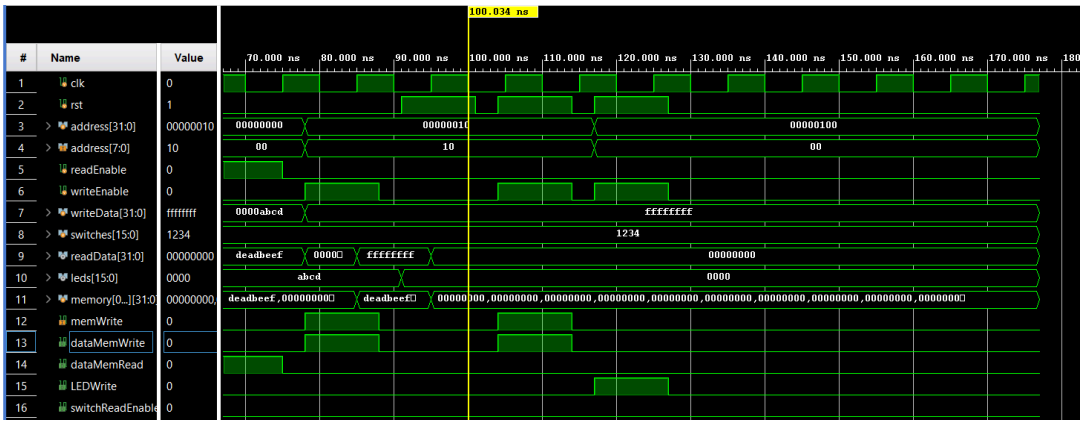
```
        #10 rst = 0;

        // Test 9: Write to DM while reset is high — write must be blocked
        #3  rst = 1; address = 32'h00000010; writeData = 32'hFFFFFFFF; writeEnable
= 1;
        #10 rst = 0; writeEnable = 0;

        // Test 10: Write to LEDs while reset is high — write must be blocked
        #3  rst = 1; address = 32'h00000100; writeData = 32'hFFFFFFFF; writeEnable
= 1;
        #10 rst = 0; writeEnable = 0;

        #50 $finish;
    end

    // Print signal values every time any monitored signal changes
    initial begin
        $monitor("Time=%0t rst=%b addr=%h we=%b re=%b wdata=%h rdata=%h
leds=%h",
                $time, rst, address, writeEnable, readEnable, writeData, readData, leds);
    end
endmodule
```

Waveforms:

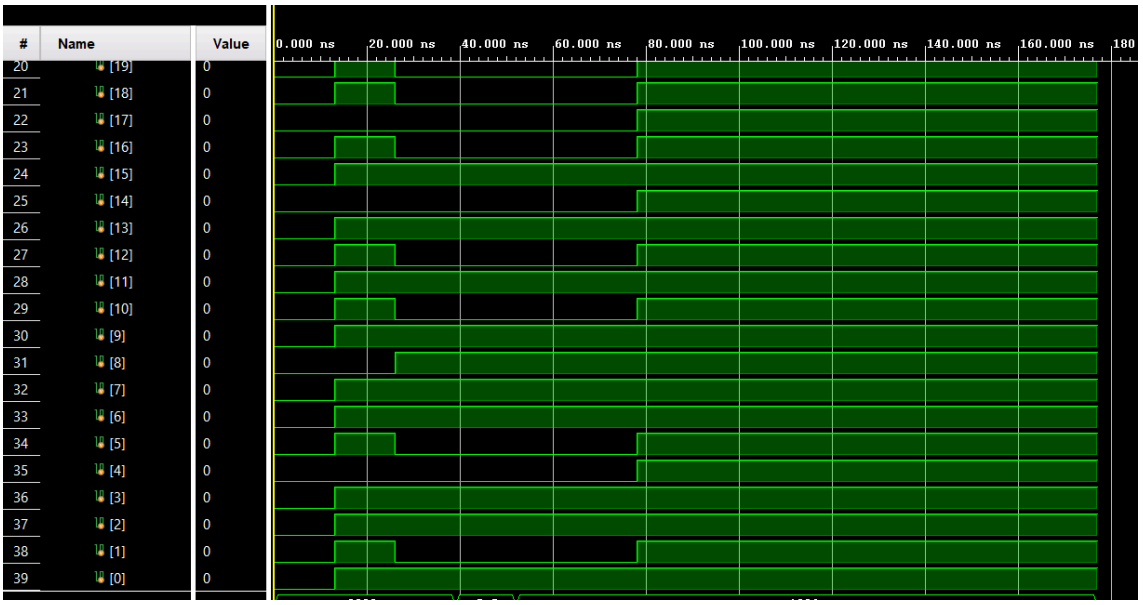
0-100 ns



100ns-170 ns

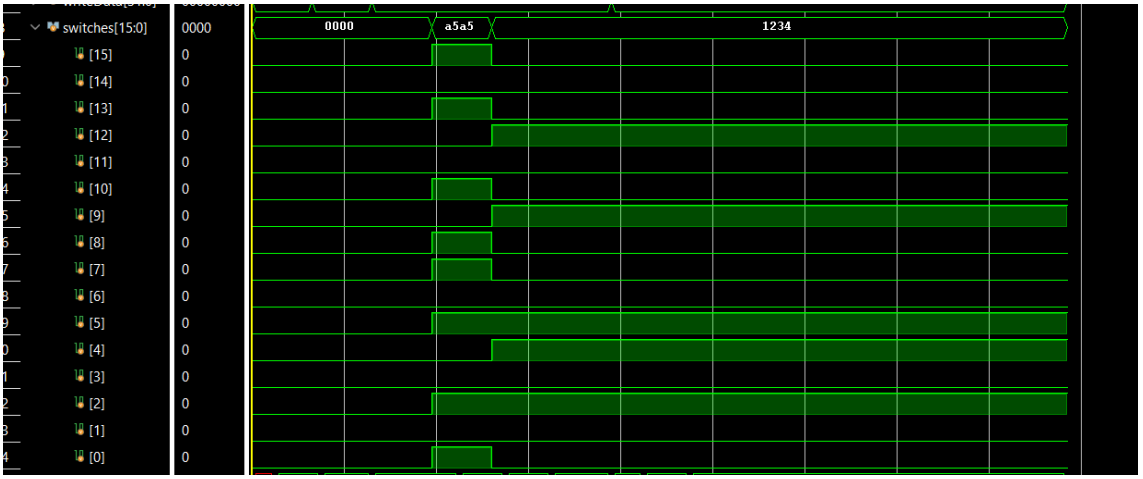


Waveform — writeData[31:0] Bit Expansion

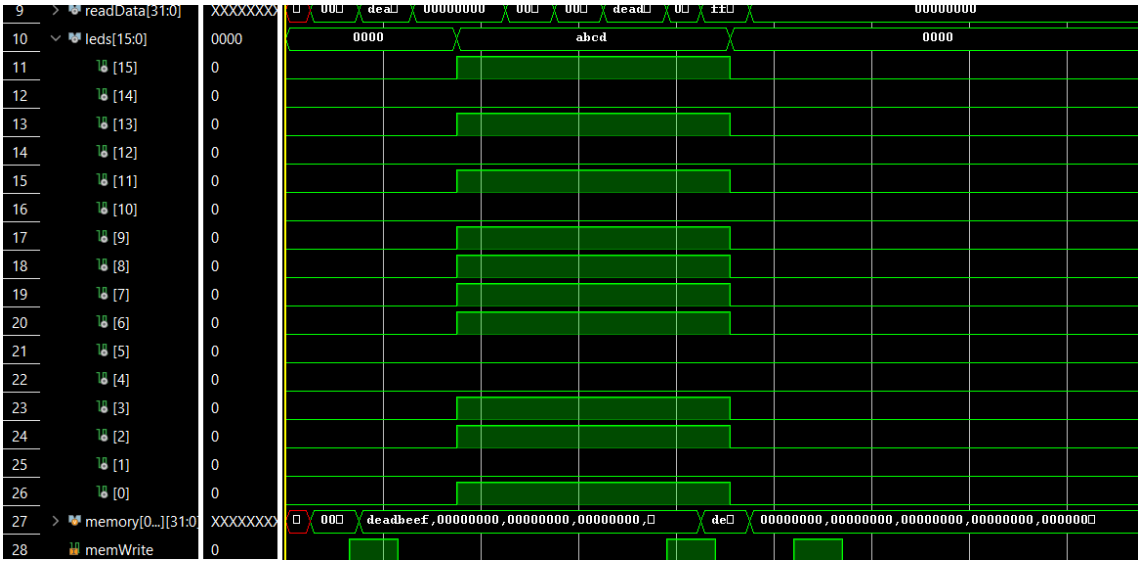


Waveform — readData[31:0] Bit Expansion

Waveform — switches[15:0] Bit Expansion



Waveform — leds[15:0] Bit Expansion



Explanations:

0–20ns — Reset

- `rst = 1`, all outputs zeroed
- `readData = 0`, `leds = 0`, memory all zeros
- Reset releases at ~20ns

Test 1: Write data memory

- `address = 0x00000000`, `writeData = 0xDEADBEEF`, `writeEnable = 1`
- `dataMemWrite` pulses high — decoder correctly selected data memory (`address[9:8] = 00`)
- `memory[0]` updates to `deadbeef` (visible in row 11)
- `readData` shows `deadbeef` immediately — asynchronous read

Test 4a: Write LEDs

- `address = 0x00000100`, `writeData = 0x0000ABCD`, `writeEnable = 1`
- `LEDWrite` pulses high — decoder selected LED interface (`address[9:8] = 01`)
- `leds` updates to `abcd` (row 10)

- `dataMemWrite` stays low — data memory unaffected ✓

Test 5a: Read switches (0xA5A5)

- `address = 0x00000200`, `switches = 0xA5A5`, `readEnable = 1`
- `switchReadEnable` pulses high — decoder selected switches (`address[9:8] = 10`)
- `readData` shows `0000a5a5`

Test 5b: Read switches (0x1234)

- Same address, `switches` changed to `0x1234`
- `readData` updates to `00001234` — confirms live switch value is captured

Test 6: Cross-talk check — read data memory

- `address = 0x00000000`, `readEnable = 1`
- `readData` returns `deadbeef` — unchanged after LED write in Test 4a
- Proves LED write did not affect data memory

Test 7: Write different DM address

- `address = 0x00000010`, `writeData = 0xFFFFFFFF`, `writeEnable = 1`
- `dataMemWrite` pulses high, `memory[0x10]` updated
- `leds` stays `abcd` — DM write did not affect LEDs

Test 8: Reset

- `rst = 1` briefly — `leds` returns to `0000`, memory cleared
- `readData` goes to `00000000`

90–100ns — Tests 9 & 10: Reset blocks writes

- `rst = 1` while `writeEnable = 1` for both DM and LED addresses
- Neither `dataMemWrite` nor `LEDWrite` cause any update — reset takes priority

Task 3

FSM

Top-Level FPGA Module — top_lab8

`top_lab8` is the top-level FPGA module. It debounces the reset button and `sw[15]` trigger, then uses a 5-state FSM to control memory operations. Toggling `sw[15]` starts a transaction — `sw[13]` selects write or read mode, `sw[12:8]` selects the memory address, and `sw[7:0]` holds the data to write. After a write, the `SHOW_LEDS` state displays the written value on the LEDs. After a read, a `WAIT_READ` state holds the address for one extra cycle to let `readData` settle before it is captured and displayed on the LEDs.

FSM AND TOP (explanation)

Sequential Operation The FSM operates step-by-step across clock cycles. Each state performs one specific task before moving to the next, ensuring operations never overlap or conflict.

Startup and Triggering The system always begins in the IDLE state and waits for a valid trigger. Since `sw[15]` is a physical switch, it is assumed to be noisy. A debouncer removes this noise and an edge detector generates a single clean one-cycle pulse, ensuring the FSM advances exactly once per toggle regardless of how long the switch is held.

Mode Selection `sw[13]` determines what the FSM does after triggering. `0` selects write mode — the immediate data on `sw[7:0]` is written to memory. `1` selects read mode — data is retrieved from the address specified by `sw[12:8]`.

Write Operation The FSM extracts the address from `sw[12:8]` and data from `sw[7:0]`, writes the data into data memory, then displays the written value on the LEDs.

Read Operation The FSM uses `sw[12:8]` as the memory address and asserts a read. A dedicated `WAIT_READ` state holds the address for one extra clock cycle to allow `readData` to stabilise. The result is then captured into a buffer register and displayed on the LEDs.

Buffer Register A `captured_read` register stores the data read from memory so it remains stable and available when the FSM reaches the `SHOW_LEDS` state, even after the read signals have been de-asserted.

LED Output LEDs serve as the visual output. In write mode they display the immediate switch data. In read mode they display the value retrieved from memory, allowing the user to verify correct storage and retrieval.

Reset Behaviour The reset signal is assumed active-high and synchronous to the clock. It forces the FSM back to IDLE and clears all internal registers including the captured read buffer, ensuring a clean restart.

Address Formation The FSM constructs a full 32-bit address from the switch inputs, placing `sw[12:8]` into `address[12:8]` with `address[9:8] = 00` to select data memory, and using `0x00000100` to target the LED interface. This directly aligns with the address decoder mapping defined in Task 2.

Synchronous Operation All state transitions, register updates, and data captures occur on the rising edge of the clock, ensuring predictable and glitch-free behaviour across all operations.

Switch Input Format The 16 switches are divided into fixed fields:

sw[15]	sw[14]	sw[13]	sw[12:8]	sw[7:0]
Trigger	—	Mode	Memory address	Data to write

```
`timescale 1ns / 1ps

// Top-level FPGA module for Lab 8
// Implements an FSM that lets the user write/read data memory and display
// results on LEDs using physical switches as input
module top_lab8(
    input clk,            // 100 MHz system clock
    input btn_reset,      // active-high reset button
    input [15:0] sw,      // switch inputs — see layout below
    output [15:0] led     // LED outputs
);
    // Switch layout: [Trigger[15] | Mode[13] | Mem Address[12:8] | Imm[7:0]]
    // sw[15] = trigger (toggle to start), sw[13] = mode (0=write, 1=read)
    // sw[12:8] = memory address, sw[7:0] = data to write

    // Debounce reset button and sw[15] trigger to avoid glitches
    wire clean_reset;
    debouncer db(
        .clk(clk),
        .pbin(btn_reset),
        .pbout(clean_reset)
    );
```

```
wire clean_sw15;
debouncer db1(
    .clk(clk),
    .pbin(sw[15]),
    .pbout(clean_sw15)
);

// Edge detector on sw[15] — generates a single-cycle pulse on each toggle
reg sw15_delay;
wire trigger_pulse;
always @(posedge clk) begin
    if (clean_reset)
        sw15_delay <= 1'b0;
    else
        sw15_delay <= clean_sw15;
end
assign trigger_pulse = clean_sw15 & ~sw15_delay;

// Memory system signals
reg [31:0] address;
reg readEnable, writeEnable;
reg [31:0] writeData;
wire [31:0] readData;

// Instantiate memory system - note updated port name switches_in / leds_out
AddressDecoderTOP mem_sys(
    .clk(clk),
    .rst(clean_reset),
    .address(address),
    .readEnable(readEnable),
    .writeEnable(writeEnable),
    .writeData(writeData),
    .switches_in(sw),
    .readData(readData),
    .leds_out(led)
);

// FSM state encoding
localparam IDLE      = 3'b000;  // wait for trigger
localparam WRITE_DM  = 3'b001;  // write sw[7:0] to data memory
localparam READ_DM   = 3'b010;  // assert read request
localparam WAIT_READ = 3'b011;  // extra cycle for readData to settle
localparam SHOW_LEDS = 3'b100;  // write result to LED interface

reg [2:0] state, next_state;
```

```
// State register — advances on every clock edge
always @(posedge clk) begin
   if (clean_reset)
      state <= IDLE;
   else
      state <= next_state;
end

// Capture readData once it has settled (one cycle after read is asserted)
reg [31:0] captured_read;
always @(posedge clk) begin
   if (clean_reset)
      captured_read <= 32'b0;
   else if (state == WAIT_READ)
      captured_read <= readData;
end

// Combinational next-state and output logic
always @(*) begin
   next_state  = state;
   address     = 32'b0;
   readEnable  = 0;
   writeEnable = 0;
   writeData   = 32'b0;

   case (state)
      IDLE: begin
         // Wait for trigger — sw[13] selects write or read mode
         if (trigger_pulse)
            next_state = (sw[13] == 0) ? WRITE_DM : READ_DM;
      end

      WRITE_DM: begin
         // Write sw[7:0] to data memory at address sw[12:8]
         address     = {22'b0, 2'b00, 3'b0, sw[12:8]};
         writeData   = {24'b0, sw[7:0]};
         writeEnable = 1;
         next_state  = SHOW_LEDS;
      end

      READ_DM: begin
         // Assert address and readEnable so memory latches the read
         address    = {22'b0, 2'b00, 3'b0, sw[12:8]};
         readEnable = 1;
```

```
            next_state = WAIT_READ;
        end

        WAIT_READ: begin
            // Hold address/readEnable one more cycle so readData is valid
            address    = {22'b0, 2'b00, 3'b0, sw[12:8]};
            readEnable = 1;
            next_state = SHOW_LEDS;
        end

        SHOW_LEDS: begin
            // Write result to LED interface (address[9:8]=01 selects LEDs)
            // Write mode: show sw[7:0] — Read mode: show captured memory value
            address     = 32'h00000100;
            writeData   = (sw[13] == 0) ? {24'b0, sw[7:0]} : captured_read;
            writeEnable = 1;
            next_state  = IDLE;
        end

        default: next_state = IDLE;
      endcase
   end
endmodule
```

Constraints File:

```
## Clock signal (100 MHz)
set_property PACKAGE_PIN W5 [get_ports clk]

   set_property IOSTANDARD LVCMOS33 [get_ports clk]
   create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]

## Switches
set_property PACKAGE_PIN V17 [get_ports {switches_in[0]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {switches_in[0]}]
set_property PACKAGE_PIN V16 [get_ports {switches_in[1]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {switches_in[1]}]
set_property PACKAGE_PIN W16 [get_ports {switches_in[2]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {switches_in[2]}]
```

```
set_property PACKAGE_PIN W17 [get_ports {switches_in[3]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {switches_in[3]}]
set_property PACKAGE_PIN W15 [get_ports {switches_in[4]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {switches_in[4]}]
set_property PACKAGE_PIN V15 [get_ports {switches_in[5]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {switches_in[5]}]
set_property PACKAGE_PIN W14 [get_ports {switches_in[6]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {switches_in[6]}]
set_property PACKAGE_PIN W13 [get_ports {switches_in[7]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {switches_in[7]}]
set_property PACKAGE_PIN V2 [get_ports {switches_in[8]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {switches_in[8]}]
set_property PACKAGE_PIN T3 [get_ports {switches_in[9]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {switches_in[9]}]
set_property PACKAGE_PIN T2 [get_ports {switches_in[10]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {switches_in[10]}]
set_property PACKAGE_PIN R3 [get_ports {switches_in[11]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {switches_in[11]}]
set_property PACKAGE_PIN W2 [get_ports {switches_in[12]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {switches_in[12]}]
set_property PACKAGE_PIN U1 [get_ports {switches_in[13]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {switches_in[13]}]
set_property PACKAGE_PIN T1 [get_ports {switches_in[14]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {switches_in[14]}]
set_property PACKAGE_PIN R2 [get_ports {switches_in[15]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {switches_in[15]}]
## LEDs
# Data Patterns (Bottom 8 LEDs)
set_property PACKAGE_PIN U16 [get_ports {leds_out[0]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {leds_out[0]}]
```

```
set_property PACKAGE_PIN E19 [get_ports {leds_out[1]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {leds_out[1]}]
set_property PACKAGE_PIN U19 [get_ports {leds_out[2]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {leds_out[2]}]
set_property PACKAGE_PIN V19 [get_ports {leds_out[3]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {leds_out[3]}]
set_property PACKAGE_PIN W18 [get_ports {leds_out[4]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {leds_out[4]}]
set_property PACKAGE_PIN U15 [get_ports {leds_out[5]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {leds_out[5]}]
set_property PACKAGE_PIN U14 [get_ports {leds_out[6]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {leds_out[6]}]
set_property PACKAGE_PIN V14 [get_ports {leds_out[7]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {leds_out[7]}]

# State Debug (Top 3 LEDs)
set_property PACKAGE_PIN V13 [get_ports {leds_out[8]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {leds_out[8]}]
set_property PACKAGE_PIN V3 [get_ports {leds_out[9]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {leds_out[9]}]
set_property PACKAGE_PIN W3 [get_ports {leds_out[10]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {leds_out[10]}]
set_property PACKAGE_PIN U3 [get_ports {leds_out[11]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {leds_out[11]}]
set_property PACKAGE_PIN P3 [get_ports {leds_out[12]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {leds_out[12]}]
set_property PACKAGE_PIN N3 [get_ports {leds_out[13]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {leds_out[13]}]
set_property PACKAGE_PIN P1 [get_ports {leds_out[14]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {leds_out[14]}]
```

```
set_property PACKAGE_PIN L1 [get_ports {leds_out[15]}]

   set_property IOSTANDARD LVCMOS33 [get_ports {leds_out[15]}]
## Buttons
# Center Button for Reset
set_property PACKAGE_PIN U18 [get_ports rst]

   set_property IOSTANDARD LVCMOS33 [get_ports rst]
# Top Button for Start
set_property PACKAGE_PIN T18 [get_ports start]

   set_property IOSTANDARD LVCMOS33 [get_ports start]
```

RTL Schematic:
PDF ATTACHED

RTL Schematic Analysis

Overall Structure

The schematic is divided into two main regions. The left side contains the FSM logic from `top_lab8.v`, and the right side contains the `AddressDecoderTOP` block which holds all the memory and peripheral sub-modules.

Left Side — FSM Logic

Address and WriteData MUXes Two multiplexers handle the combinational output logic of the FSM. Each MUX has one input per FSM state and uses the current state register as its select line. Whichever state is active determines which value passes through.

The `address` MUX selects between `32'b0` in IDLE, the data memory address `{00, sw[12:8]}` during write and read states, and `0x00000100` during SHOW_LEDS to target the LED interface.

The `writeData` MUX selects between `32'b0` in IDLE, `{24'b0, sw[7:0]}` during WRITE_DM, and either `sw[7:0]` or `captured_read` during SHOW_LEDS depending on the mode.

Enable Signal Logic The `readEnable` and `writeEnable` signals are synthesized as small lookup tables by Vivado. Each state maps to either a 0 or 1 — for example

`writeEnable` is only 1 in WRITE_DM and SHOW_LEDS, so Vivado implements this as a simple truth table driven by the state register.

State Registers Five flip-flops implement the FSM using one-hot encoding — one flip-flop per state. Only one is ever high at a time. Small LUT blocks between them implement the next-state logic. A single additional flip-flop implements the `sw15_delay` edge detector.

Captured Read Register Eight flip-flops store the data read from memory during `WAIT_READ`. This keeps the value stable and available when the FSM reaches `SHOW_LEDS`, even after the read signals have been de-asserted.

Right Side — AddressDecoderTOP Block

Address Decoder (`ad`) Takes `address[9:8]` along with `readEnable` and `writeEnable`, and produces `dataMemWrite`, `dataMemRead`, `LEDWrite`, and `switchReadEnable`. This is purely combinational logic — no registers, synthesized as simple AND gates and comparators.

Data Memory (`dataMem`) Takes an 8-bit address, `writeData`, `memWrite`, `clk`, and `rst`. The `+` symbol on the block indicates Vivado inferred this as Block RAM — a dedicated hardware memory resource on the Artix-7, giving efficient storage for all 512 × 32-bit locations.

LED Interface (`led_inst`) Contains a 16-bit register that latches `writeData[15:0]` when `LEDWrite` is asserted. The `+` symbol indicates internal flip-flops. Its output drives the physical LED pins directly.

Switch Interface (`sw_inst`) Contains a 32-bit register that latches `{16'b0, switches}` when `readEnable` is asserted. The `+` symbol indicates internal registers. This is why the `WAIT_READ` state is needed — the registered output takes one clock cycle to update.

ReadData MUX The `assign readData` statement in `AddressDecoderTOP` is synthesized as two cascaded 2-to-1 multiplexers. The first selects between data memory output and `32'b0`, and the second selects between that result and the switch output, both controlled by `address[9:8]`.

Signal Flow — Write Operation

sw[15] toggle → trigger pulse → FSM enters WRITE_DM
→ address MUX outputs {00, sw[12:8]}
→ writeData MUX outputs {0, sw[7:0]}
→ writeEnable asserted
→ AddressDecoder sees address[9:8]=00 → dataMemWrite goes high
→ DataMemory stores writeData at address[7:0]
→ FSM moves to SHOW_LEDS
→ address MUX outputs 0x100 → LEDWrite goes high
→ LED register latches writeData[15:0]
→ Physical LEDs update

Signal Flow — Read Operation

sw[15] toggle → trigger pulse → FSM enters READ_DM
→ address MUX outputs {00, sw[12:8]}
→ readEnable asserted
→ AddressDecoder sees address[9:8]=00 → dataMemRead goes high
→ DataMemory outputs readData combinationally
→ FSM moves to WAIT_READ — holds address and readEnable one more cycle
→ Switch/memory readData now stable
→ captured_read register latches readData
→ FSM moves to SHOW_LEDS
→ writeData MUX selects captured_read
→ address MUX outputs 0x100 → LEDWrite goes high
→ LED register latches captured_read value
→ Physical LEDs display the retrieved memory value

Lab 08 – Design and FPGA Implementation of Memory System with Address Decoding for a Single-Cycle RISC-V Processor

Points Distribution

Task No.	LR2 Code	LR 5 Results	LR11 Design	AR7
Task 1	-	-	/20	/20
Task 2	/20	/15		
Task 3	/10	/15		
Total Points	/80			/20
CLO Mapped	CLO 1			

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3:Good Points 3	Level 4:Exemplary Points 4
LR2	Program/Code / Simulation Model/ Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
AR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all

			Report is submitted after due date.	The report is submitted timely.	the necessary figures, plots and tables.